

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	[팀명 입력 필요 - 참가 접수 정보와 대조]	팀 인원 (팀장 포함)	[N]명
참가부문	일반	과제유형	자유과제

□ 결과보고서

프로젝트 개요	
프로젝트명	SongRyeon Core v1 (접수 작품명과 최종 대조 필요)
프로젝트 등록 URL	https://github.com/Junghoo-developer/SongRyeon/tree/codex/contest-release-2026
시연영상	[3분 이하 YouTube 시연 영상 URL 입력 필요]
프로젝트 소개	코드가 확인한 실행 사실(A)과 LLM이 만든 판단(R)을 원자 기록으로 분리하고, 증거 수집·충분성 검사·답변·검열을 네 노드로 나눈 로컬 오픈 웨이트 에이전트 런타임.
프로젝트 세부 내용	
개발배경 및 목적	문제 LLM 에이전트는 실제로 실행한 도구·읽은 파일과 모델이 추론한 내용을 같은 문맥에서 섞기 쉽다. 이때 모델이 실행 동선까지 잘못 말하면 사용자는 답변의 근거를 추적하기 어렵다. 목표 세상 모든 문장의 진위를 판정하는 대신, 코드가 그대로 확인할 수 있는 실행 사실과 모델의 해석을 저장·공개·검열 단계에서 분리한다. 모델의 모든 서술은 R로, 언제 어느 노드가 무엇을 실행해 어떤 원문을 얻었는지는 A로 남긴다.
개발환경	언어·저장 Python 3.10 이상, UTF-8 JSONL 원본 기억, SQLite 지식 색인. 실행부는 필수 외부 Python 패키지 없이 표준 라이브러리로 구성했다. 모델 실행 Ollama 0.32.5의 로컬 HTTP 인터페이스. 제출 프로파일은 gemma4:26b(Q4_K_M, 컨텍스트 16,384, temperature 0)이며, 공식 v2는 동일 모델의 Node2·Node4 없는 단일 에이전트·Node4 제외·전체 송편을

	비교했다. seed는 1709, 2647, 3911이다. 검증 환경 Windows 11 Pro 64-bit, Intel Core i9-14900K, NVIDIA GeForce RTX 5080 16,303 MiB, 메모리 63.69 GiB, Python 3.10.11. 최소 사양이 아니라 2026-07-30에 관찰한 호스트 기록이다.
시스템 구성 및 아키텍처	요청 → Node1 → Node2 → Node3 → Node4 → 최종 답변 Node1만 프로젝트 경계 안의 읽기 전용 Python 파일 도구를 사용한다. 도구 원문은 감사 로그에 보존되고, Node1이 full-excerpt-결정론적 chunk-omit 중 공개 방식을 고르면 코드는 원문에서 선택 구간을 그대로 복사한다. Node2는 요청에 필요한 A가 충분한지 검사하고, Node3는 사용자 답변을 작성하며, Node4는 답변의 현재 코드 주장을 공개된 tool_result_content A와 대조한다. Node2·Node4는 각각 반려 3회 상한을 가지며 상한 초과는 permit이 아니라 검증 미완료 상태로 표시된다. 원본 기억 7필드: information, information_class, code_verifiable, information_type, turn_id, information_id, created_at. 에이전트 시야는 숨김 원문과 UUID·시각을 제외한 원자에 원본 줄 기반 memory_index를 붙인다. 턴 시작 시 잡은 가장 오래된 공개 원자를 턴 종료까지 기준점으로 고정한다.
프로젝트 주요기능	핵심 기능 1) append-only JSONL에 A/R 원자와 노드·도구 동선을 기록한다. 2) 모델이 고른 도구 원문 구간은 코드가 exact copy한다. 3) 경로 이탈·심볼릭 링크 이탈·비 Python 파일·크기 초과를 거부한다. 4) Node2의 증거 충분성 검사와 Node4의 답변 근거 검열을 분리한다. 코드 검증 공개 결과 커밋 33c9453을 GitHub에서 새 폴더로 clone하고 새 가상환경에 설치해 276 passed, 2 skipped를 재현했다. 두 skip은 Windows 심볼릭 링크 생성 권한이 없는 환경의 경계 시험이다. demo 도움말·fixture 평가·공개 결과 검증·wheel 빌드와 새 환경 설치도 통과했다. 공식 v2 holdout 24개 합성 Python 사례 × 3개 구조 × 3개 seed = 216회. 단일 에이전트 71/72(1건 ModelResponseError로 미완료), Node4 제외 승련 72/72, 전체 승련 72/72. 세 구조의 unsupported→supported 기계 proxy는 0건이다. 평가 무결성과 경계 첫 출력 전 소스·case·채점 규칙과 HMAC commitment를 동결했다. 첫 실행의 채점기 결함은 무효화하고, 같은 case·같은 SUT를 교정한 프로토콜로 재실행했다. v2 정제 결과의 공개 해시 검증은 통과했지만 필수 19개 사람 설명 감사 전 publishable=false다. 두 승련 구조가 동물이므로 Node4 효과·환각 제거·통계

	적 유의성·업무 일반화를 주장하지 않는다.
기대효과 및 활용분야	기대 효과 코드 조사·리뷰형 에이전트에서 실제로 읽은 A와 모델이 판단한 R을 분리해 출처와 실행 동선을 다시 추적할 수 있다. 실패도 같은 형식으로 남겨 재현·감사·개선 대상을 좁힌다. 이번 실험이 준 설계 판단 전체 송련과 Node4 제외 구조의 독립 end-to-end verdict가 72쌍 모두 같았다. 따라서 이 쉬운 합성 집합에서는 Node4 상시 호출의 추가 이득을 확인하지 못했다. 다음에는 더 어려운 경계 사례와 선택적 Node4 정책의 비용 대비 효과를 측정한다. 활용·확장 읽기 전용 코드 조사와 내부 문서 감사에 적용하고, 이후 지식 DB·과거 기억 검색과 원본 보존형 파생 시야 압축을 같은 A/R 계약 아래 확장한다.
기타	차별성 더 큰 프롬프트 하나에 신뢰를 맡기지 않는다. 모델 출력은 R로 유지하고, 도구 원문·카운터·라우팅은 코드가 A로 기록한다. 반려 한도 초과도 성공으로 위장하지 않고 검증 미완료로 표시한다. 현재 한계 도구는 .py 목록·열람에 집중하며 기억 검색·압축은 없다. v2는 첫 verdict만 기계 채점했고 19개 감사는 216개 전체 설명 검증이 아니다. 공개 verifier는 정제 결과의 해시 정합성을 검사하지만 private raw 재생과 lock/reveal의 별도 공개 시점은 입증하지 못한다. 다음 실험은 두 공개 커밋을 분리한다. 재현 기준 실험 소스 동결 8e37b4c303081afcfa23ec0ef730e02b2f27ffa6, 공개 결과 체인 33c94534c168fb015b85984832503a6de5f170c6. 제출 최종 SHA·영상 URL·신원 정보는 동결 뒤 입력한다. 개발 소감 [참가자가 직접 작성: 문제 정의, A/R 분리를 구현하며 겪은 시행착오, 실제 데모와 테스트에서 배운 점을 본인의 경험으로 3~5문장 기재]

붙임1

SBOM[소프트웨어 자재명세서]

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	Python / 표준 라이브러리	3.10.11	PSF-2.0	https://www.python.org/	애플리케이션 실행, JSON·SQLite·HTTP·경로 처리
2	setuptools	65.5.0	MIT	https://github.com/pypa/setuptools	PEP 517 빌드 backend
3	pytest	9.1.1	MIT	https://github.com/pytest-dev/pytest	선택적 개발·회귀 테스트
4	Ollama	0.32.5	MIT	https://github.com/ollama/ollama	로컬·자체 호스팅 모델 HTTP 실행
5	Gemma 4 26B A4B	5571076f3d70 / Q4_K_M	Apache-2.0	https://ollama.com/library/gemma4:26b	대회 제출 프로필의 로컬 생성 모델
6	Qwen3-14B	bdbd181c33f2 / Q4_K_M	Apache-2.0	https://huggingface.co/Qwen/Qwen3-14B	과거 로컬 탐색 기준선(공식 v2 비교군·fallback 아님)

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☒ 표시)

☒ 유형 1: 외부 공개 모델을 추가 학습 없이 프로젝트에 연동·구동

☐ 유형 2: 외부 모델 파인튜닝

☐ 유형 3: 자체 개발 모델

개발 과정의 상용 AI 코딩 보조 사용은 아래 4번 항목에 별도 공개하며, 실행 모델의 유형 선택과 구분한다.

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	Gemma 4 26B A4B (Google DeepMind), Ollama tag gemma4:26b, 25.8B parameters, Q4_K_M; digest 5571076f3d70050487b26b341705799e0ab29b808164f90d20d4cf84f699d251	기반 모델 라이 선스	Apache License 2.0 (https://ollama.com/library/gemma4:26b)
-----------------	---	----------------	---

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.
데이터 정제/ 가공 방법 요 약	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.
새로 생성된 가중치 공개 저장소	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.

URL			
가중치 파일 정보 및 배포방식	해당 없음 - 유형 1로 추가 학습·파인튜닝을 수행하지 않았으며 학습 데이터셋이나 새 가중치를 생성하지 않음.		
4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)			
직접 작성한 코드의 오픈소스 라이 선스	MIT License (저장소 LICENSE 포함)	학습/추론 소스 코드 공개 저장소 URL	https://github.com/Junghoo-developer/SongRyeon/tree/codex/contest-release-2026
상용 AI 보조 도구 활용 여 부 및 범위	OpenAI Codex가 구조·프롬프트 토론, 코드·테스트·평가 도구·문서의 작성과 실행을 광범위하게 수행했다. 참가자는 문제와 A/R 철학, 노드 역할, 실험 질문을 정하고 실제 모델 출력·프롬프트·주요 코드 흐름을 반복 검수해 변경 채택과 최종 제출 주장을 직접 승인한다. 일부 생성 코드는 회귀 테스트 뒤 출품 소스에 포함됐다.		